Apunte de JavaScript y TypeScript

Material complementario - Clase #05

Sobre este documento

Sirve como referencia rápida para que tengan a mano cuando necesiten consultar la sintaxis o entender qué hace cada cosa. No reemplaza la documentación oficial (MDN, TypeScript Handbook), pero sí condensa lo que vamos a usar todo el resto de la cursada.

Los temas que vienen acá son los que necesitan dominar antes de entrar a React. Cuando se sientan medio perdidos en una clase de React, lo más probable es que el tema esté acá: cómo se importa un archivo, cómo se desestructura un objeto, cómo se espera una respuesta del servidor.

¿Cómo leer este apunte?

Si vienen siguiendo la cursada, leer todo de corrido la primera vez (~30 min).

Cada tema tiene la motivación (“Por qué importa”) y la sintaxis (“Cómo se usa”).

Los ejemplos están tipados con TypeScript desde el principio. JavaScript puro lo verán solo cuando hace falta para entender la diferencia.

Al final hay una guía de los 4 ejercicios prácticos de la clase.

Lo que ya saben

- De C03 (JS básico): variables, condicionales, funciones, arrays, bucles, manipulación básica del DOM.
- De C04 (TS intro): tipos básicos, funciones tipadas, interfaces, clases con interfaces.

Índice

Sobre este documento 1
Índice 2
1. Módulos: import / export 3
Named exports 3
Default export 4
Renombrar al importar 4
2. Arrow Functions 5
Forma tradicional vs flecha 5
Reglas básicas 5
Uso típico: callbacks de array 5
3. Destructuring 6
Destructuring de objetos 6
Renombrar al desestructurar 6
Valores por defecto 6
Destructuring de arrays 7
Destructuring en parámetros de función 7
4. Spread / Rest: el operador 8
Spread: esparcir un array u objeto 8
Rest: juntar varios valores en uno 9
5. Promises: operaciones asincrónicas 10
Forma original: .then() / .catch() 10
6. async / await: la forma moderna 11
7. Fetch API: pedir datos al servidor 12
Forma básica 12
¿Qué devuelve fetch? 12
Tipar la respuesta en TypeScript 12
POST: enviar datos 13

1. Módulos: import / export

*Hasta ahora, todo el código lo teníamos en un mismo archivo (o como mucho dos). Eso **no escala**: cuando un proyecto crece, necesitamos partirlo en archivos chicos, cada uno con una responsabilidad clara, y poder usar lo que está en un archivo desde otro.*

Los módulos son la forma estándar de hacer eso. Se exportan funciones, clases, tipos o constantes desde un archivo, y se importan desde otros.

> **Antes de empezar**
> Para que esto funcione, el archivo HTML que carga el JS principal debe tener **type="module"**:
> `<script type="module" src="./app.js"></script>`
> Sin eso, los navegadores no entienden import/export.

Named exports

Se exportan elementos con nombre. Un archivo puede tener varios exports.

```typescript
// utils.ts
export const PI = 3.14159;

export function areaCirculo(radio: number): number {
  return PI * radio * radio;
}

export interface Libro {
  titulo: string;
  autor: string;
}
```

Y desde otro archivo se importan:

```typescript
// app.ts
import { PI, areaCirculo, Libro } from './utils';

console.log(PI);
console.log(areaCirculo(5));

const miLibro: Libro = {
  titulo: 'Rayuela',
  autor: 'Cortázar'
};
```

Default export

Un archivo puede tener **UN solo export default**. Es para "lo principal" del archivo.

```typescript
// libro.ts
export default class Libro {
  constructor(public titulo: string, public autor: string) {}
}
```

```typescript
// app.ts
import Libro from './libro';   // sin llaves
const l = new Libro('1984', 'Orwell');
```

> **Cuándo usar cada uno**
>
> **Named export:** cuando el archivo tiene varias cosas para exportar (utilidades, helpers, tipos relacionados).
>
> **Default export**: cuando el archivo expone una sola cosa principal (un componente, una clase). En React lo van a usar muchísimo para componentes.
>
> **Pueden combinarse**: un default + varios named en el mismo archivo.

Renombrar al importar

A veces dos archivos exportan algo con el mismo nombre, o queremos usar otro alias:

```typescript
import { areaCirculo as area } from './utils';

console.log(area(5));
```

2. Arrow Functions

Las arrow functions (funciones flecha) son una forma más corta y moderna de escribir funciones. Son las que más van a ver en React: cualquier handler de evento, cualquier callback, casi todo se escribe con flechas.

Forma tradicional vs flecha

Forma Tradicional	Arrow Function
`function suma(a: number, b: number): number { return a + b; }`	`const suma = (a: number, b: number): number => a + b;`
`function saludar(nombre: string) { console.log('Hola ' + nombre); }`	`const saludar = (nombre: string) => { console.log('Hola ' + nombre); };`

Reglas básicas

- Si hay un solo parámetro, los paréntesis son opcionales: `n => n * 2`
- Si el cuerpo es una sola expresión, no hace falta llaves ni return: `(a, b) => a + b`
- Si hay varias líneas, sí hacen falta llaves y `return` explícito.
- En TypeScript, los tipos van en los parámetros y opcionalmente en el retorno: `(n: number): string => `Hola ${n}``

Uso típico: callbacks de array

Acá es donde más las van a ver. Métodos como map, filter, find, forEach reciben funciones como argumento, y casi siempre se pasan como flechas:

```typescript
const numeros: number[] = [1, 2, 3, 4, 5];

// Duplicar cada número
const dobles = numeros.map(n => n * 2);                    // [2, 4, 6, 8, 10]

// Filtrar pares
const pares = numeros.filter(n => n % 2 === 0);     // [2, 4]

// Encontrar el primero mayor a 3
const primero = numeros.find(n => n > 3);           // 4
```

> **Cuándo NO usar arrow functions**
>
> Cuando necesitan un método de una clase, sigan usando la sintaxis tradicional: las arrows tienen un comportamiento distinto con "this" y pueden traer dolores de cabeza dentro de clases.
>
> Para funciones de primer nivel reutilizables, ambas formas funcionan, la elección es de estilo.

3. Destructuring

Destructuring es "sacar cosas de adentro de un objeto o array y ponerlas en variables sueltas" en una sola línea. Ahorra muchísimo código y lo van a usar y ver todo el tiempo en React (con props, state, hooks).

Destructuring de objetos

```typescript
interface Libro {
  titulo: string;
  autor: string;
  anio: number;
}

const libro: Libro = {
  titulo: 'Rayuela',
  autor: 'Cortázar',
  anio: 1963
};

// Sin destructuring
const titulo1 = libro.titulo;
const autor1 = libro.autor;
console.log(titulo1, autor1);       // Rayuela Cortázar

// Con destructuring
const { titulo2, autor2 } = libro;
console.log(titulo2, autor2);      // El Aleph Borges
```

Renombrar al desestructurar

```typescript
const { titulo: nombreLibro, autor: escritor } = libro;
console.log(nombreLibro);          // Rayuela
```

Valores por defecto

```typescript
const { titulo, editorial = 'Desconocida' } = libro;
console.log(editorial);     // Desconocida (porque libro no la tiene)
```

Destructuring de arrays

Igual pero con corchetes y por posición:

```typescript
const colores: string[] = ['rojo', 'verde', 'azul'];

const [primero, segundo] = colores;
console.log(primero); // 'rojo'
console.log(segundo); // 'verde'

// Saltearse posiciones
const [, , tercero] = colores;
console.log(tercero); // 'azul'
```

Destructuring en parámetros de función

Esto se usa todo el tiempo. En vez de recibir un objeto y acceder a sus campos adentro, se desestructura directamente al recibirlo:

```typescript
// Sin destructuring
function mostrarLibro(libro: Libro) {
  console.log(libro.titulo + ' - ' + libro.autor);
}

// Con destructuring
function mostrarLibro({ titulo, autor }: Libro) {
  console.log(`${titulo} - ${autor}`);
}

// En ambos casos se utilizan igual
mostrarLibro(libro);
```

4. Spread / Rest: el operador ...

*El operador ... (tres puntos) sirve para dos cosas distintas según dónde lo pongan: **spread** (esparcir) o **rest** (juntar). Es clave para inmutabilidad, que es una idea central en React.*

Spread: esparcir un array u objeto

Toma todo lo que hay adentro de un array u objeto y lo "desglosa" en elementos individuales.

Con arrays

```typescript
const frutas: string[] = ['manzana', 'pera'];
const verduras: string[] = ['lechuga', 'tomate'];

// Combinar arrays
const todo: string[] = [...frutas, ...verduras];
// ['manzana', 'pera', 'lechuga', 'tomate']

// Copiar un array (¡no es la misma referencia!)
const copia: string[] = [...frutas];

// Agregar al final
const conNueva: string[] = [...frutas, 'banana'];
```

Con objetos

```typescript
const libro: Libro = {
  titulo: 'Rayuela',
  autor: 'Cortázar',
  anio: 1963
};

// Copiar un objeto y modificar un campo
const libroClonado: Libro = { ...libro, anio: 2023 };

// { titulo: 'Rayuela', autor: 'Cortázar', anio: 2023 }
// libro queda intacto
```

Rest: juntar varios valores en uno

Cuando ... aparece como parámetro de una función, hace lo opuesto: junta todos los argumentos sueltos en un array.

```typescript
function sumar(...numeros: number[]): number {
  return numeros.reduce((acc, n) => acc + n, 0);
}

console.log(sumar(1, 2, 3));           //   6
console.log(sumar(1, 2, 3, 4, 5));     //  15
```

También se usa en destructuring para “lo que sobra”:

```typescript
const [primero, ...resto] = [1, 2, 3, 4, 5];
console.log(primero); // 1
console.log(resto);   // [2, 3, 4, 5]

const { titulo, ...otros } = libro;
console.log(titulo);  // 'Rayuela'
console.log(otros);   // { autor: 'Cortázar', anio: 1963 }
```

5. Promises: operaciones asincrónicas

Hasta ahora todas sus líneas de código se ejecutaban una atrás de la otra, en orden, instantáneamente. Pero hay cosas que tardan: pedirle datos a un servidor, leer un archivo, esperar que el usuario haga click.

*Las **Promises** (promesas) permiten que Javacript no se quede congelado esperando: el programa sigue corriendo y, cuando la operación lenta termina, la promesa te avisa para retomar de donde quedó.*

Una Promise representa "un valor que va a estar listo en algún momento". Tiene tres estados:

- *pending — todavía está pasando.*
- *fulfilled — se completó OK, hay un valor.*
- *rejected — falló, hay un error.*

Forma original: .then() / .catch()

Esta sintaxis todavía la van a ver en código existente, así que hay que conocerla:

```typescript
fetch('https://api.ejemplo.com/libros')
  .then(response => response.json())
  .then(libros => console.log(libros))
  .catch(error => console.error('Falló:', error));
```

Cada .then() recibe el resultado del anterior. Si algo falla en cualquier punto, salta al .catch().

> **El problema con muchos .then() encadenados**
>
> Cuando hay 5 o 6 pasos, el código se vuelve difícil de leer: niveles y niveles de anidación, errores difíciles de seguir.
>
> Eso lo soluciona async/await.

6. async / await: la forma moderna

async/await *es una forma más simple y elegante de escribir Promises. El código se lee como si fuera secuencial, pero por debajo siguen siendo promesas. Es la forma estándar hoy de manejar cualquier cosa asincrónica.*

- **`async`**: va antes de una función. Le dice al motor de JS "esta función puede tardar y siempre devuelve una Promise".
- **`await`**: solo se puede usar dentro de una función `async`. Pausa la función hasta que la promesa se resuelva, y devuelve el valor.

El mismo ejemplo, con async/await

```typescript
async function obtenerLibros(): Promise<Libro[]> {
 try {
   const response = await fetch('https://api.ejemplo.com/libros');
   if (!response.ok) {
     throw new Error(`HTTP ${response.status}`);
   }
   const libros: Libro[] = await response.json();
   return libros;
 } catch (error) {
   console.error('Error al obtener libros:', error);
   return [];
 }
}

// Llamarla (en un módulo o función async):
const libros = await obtenerLibros();
console.log(libros);
```

Reglas de oro

- Si usan **await**, la función tiene que ser **async**.
- Toda función async devuelve una Promise, aunque ustedes solo escriban return libros.
- Para manejar errores con async/await: try/catch (no .catch()).
- Si no necesitan esperar el resultado, no pongan await: la promesa se ejecuta igual.

Comparación visual

.then() / .catch()	async / await

`fetch(url)`
`.then(r => r.json())`
`.then(data => console.log(data))`
`.catch(err => console.error(err));`
	`try {`
	`  const r = await fetch(url);`
	`  const data = await r.json();`
	`  console.log(data); }`
	`catch (err) { console.error(err); }`

7. Fetch API: pedir datos al servidor

fetch *es la función nativa de los navegadores para hacer peticiones HTTP. Le pasamos una URL, devuelve una promesa con la respuesta. Toda la cursada (front, back, integración) se va a apoyar en fetch o en variantes de fetch.*

Forma básica

```typescript
// GET básico
const response = await fetch('https://api.ejemplo.com/users');
const usuarios = await response.json();
console.log(usuarios);
```

¿Qué devuelve fetch?

Una Response. Antes de leer los datos, podemos chequear el estado:

- `response.ok: true` si el status es 2xx.
- `response.status`: el código numérico (200, 404, 500...).
- `response.json()`: parsea el cuerpo como JSON (es asincrónico, va con `await`).

Tipar la respuesta en TypeScript

Cuando hacemos `response.json()`, TS no sabe qué forma tienen los datos: por defecto los tipa como `any`. Es buena práctica decirle qué esperamos:

```typescript
interface Usuario {
  id: number;
  name: string;
  email: string;
}

async function obtenerUsuarios(): Promise<Usuario[]> {
  const response =
            await fetch('https://jsonplaceholder.typicode.com/users');
  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  const usuarios: Usuario[] = await response.json();
  return usuarios;
}
```

POST: enviar datos

Para crear, actualizar o eliminar, fetch acepta un segundo argumento con opciones:

```typescript
interface NuevoLibro {
  titulo: string;
  autor: string;
}

async function crearLibro(libro: NuevoLibro) {
  const response = await fetch('https://api.ejemplo.com/libros', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(libro)
  });
  if (!response.ok) throw new Error('Falló la creación');
  return response.json();
}
```

Cosas que se olvidan

- El **body** siempre va con **JSON.stringify**(...). Pasarle un objeto directamente no funciona.
- El header **Content-Type: application/json** es obligatorio para que el servidor sepa qué le mandamos.
- fetch NO tira excepción si el servidor responde 404 o 500: tiran excepción sólo si la red falla. Por eso siempre se **chequear response.ok**.